

Course Code: MLA0201

Name: MAGESH.S

Course Name: Fundamentals of Machine Learning

Reg.no:192525002

EXPERIMENT 1-Spam Email Prediction Using Probability Distribution

Problem Description

The objective is to build a mail-filtering system that predicts whether an incoming email is spam or legitimate. Email text is converted into numerical features using TF-IDF. Multinomial Naive Bayes applies prior and conditional probabilities to calculate the likelihood of the email belonging to the spam or non-spam class.

The model predicts:

- 0 – Not spam or legitimate email
- 1 – Spam email

Dataset Description

The **Spam Email Classification Dataset** was uploaded by **Puru Singhvi**. It combines email messages from the TREC 2007 Public Spam Corpus and the Enron-Spam dataset. It contains the columns text and label. The Kaggle description lists 83,446 labelled emails, while the downloaded CSV used during this experiment loaded with 83,448 rows.

Dataset source: [Spam Email Classification Dataset – Kaggle](#)

Algorithm / Pseudocode

1. Load the spam-email dataset from the CSV file.
2. Remove missing and duplicate email records.
3. Divide the dataset into training and testing sets.
4. Convert email text into TF-IDF numerical features.
5. Train a Multinomial Naive Bayes classifier.
6. Evaluate the model using accuracy and a confusion matrix.
7. Predict spam probability for a new email entered by the user.

Python Code

```
from pathlib import Path
```

```
import pandas as pd
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB


print("Spam Email Detection Program Started")


# Locate dataset
folder = Path(__file__).resolve().parent
dataset_path = folder / "spam-email-classification.csv"


if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
    raise SystemExit


# Load dataset
data = pd.read_csv(
    dataset_path,
    encoding_errors="ignore",
    on_bad_lines="skip"
)


print("Dataset loaded successfully")
print("Original dataset size:", data.shape)


# Select and clean columns
data = data[["text", "label"]].copy()
```

```
data = data.dropna()

data = data.drop_duplicates(subset=["text"])


data["text"] = data["text"].astype(str)
data["label"] = pd.to_numeric(
    data["label"],
    errors="coerce"
)


data = data.dropna()
data["label"] = data["label"].astype(int)
data = data[data["label"].isin([0, 1])]


print("Dataset size after cleaning:", data.shape)
print("\nClass distribution:")
print(data["label"].value_counts())


# Input and target
X = data["text"]
y = data["label"]


# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)


# Convert text into TF-IDF features
```

```
vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=10000
)

X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)

# Train Naive Bayes model
model = MultinomialNB()
model.fit(X_train_vector, y_train)

print("\nModel trained successfully")

# Evaluate model
predictions = model.predict(X_test_vector)

print(
    "Model accuracy:",
    round(accuracy_score(y_test, predictions) * 100, 2),
    "%"
)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        predictions,
```

```
        target_names=["Not Spam", "Spam"],
        zero_division=0
    )
)

# Predict a new email
new_email = input("\nEnter an email message: ")

new_email_vector = vectorizer.transform([new_email])

prediction = model.predict(new_email_vector)[0]
probability = model.predict_proba(new_email_vector)[0]

not_spam_probability = probability[0] * 100
spam_probability = probability[1] * 100

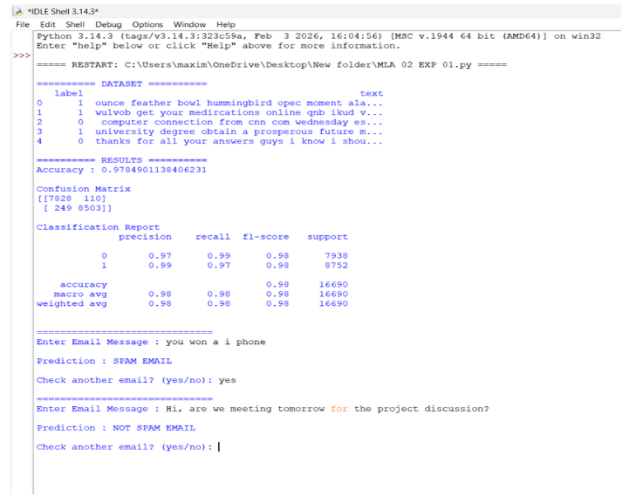
print(
    "\nNot-Spam Probability:",
    round(not_spam_probability, 2),
    "%"
)

print(
    "Spam Probability:",
    round(spam_probability, 2),
    "%"
)

if prediction == 1:
    print("Final Result: SPAM EMAIL")
else:
```

```
print("Final Result: NOT SPAM EMAIL")
```

Output Screenshot



```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\umaxim\OneDrive\Desktop\New folder\MLA 02 EXP 01.py =====

===== DATASET =====
label      text
0          1 ounce feather bowl hummingbird opec moment ala...
1          1 vulvob get your medications online qmb ikud v...
2          0 computer connection from cnn com wednesday es...
3          1 university degree obtain a prosperous future m...
4          0 thanks for all your answers guys i know i shou...

===== RESULTS =====
Accuracy : 0.9784901138406231

Confusion Matrix
[[7828 115]
 [ 249 850]]

Classification Report
      precision    recall  f1-score   support

0       0.97       0.99       0.98       7938
1       0.99       0.97       0.98       8752

accuracy      0.98      0.98      0.98      16690
macro avg     0.98      0.98      0.98      16690
weighted avg  0.98      0.98      0.98      16690

=====
Enter Email Message : you won a i phone
Prediction : SPAM EMAIL
Check another email? (yes/no): yes

Enter Email Message : Hi, are we meeting tomorrow for the project discussion?
Prediction : NOT SPAM EMAIL
Check another email? (yes/no): |
```

Result

The spam-mail filtering system was implemented successfully using TF-IDF and Multinomial Naive Bayes. The model calculated the conditional probability of an email belonging to the spam and non-spam classes and classified the email according to the class with the greater posterior probability.

EXPERIMENT 3-Student Performance Prediction Using a Learning System

Problem Description

The objective is to design a machine-learning system that predicts whether a student is likely to pass or be academically at risk. The system learns from previous student records containing attendance, study hours, previous scores, sleep hours, tutoring sessions and physical activity.

A performance label is created as:

- 0 – At Risk
- 1 – Pass

Students with an Exam_Score of 60 or above are labelled as passing.

Dataset Description

The **Student Performance Factors** dataset contains information about study habits, attendance, previous marks, sleep, tutoring, parental involvement and other factors affecting examination performance. The downloaded dataset contains 6,607 records and 20 columns, with Exam_Score as the final score.

Dataset source: [Student Performance Factors – Kaggle](#)

Algorithm / Pseudocode

1. Load the student-performance dataset.
2. Select attendance, study hours and other relevant features.
3. Clean missing, duplicate and invalid records.
4. Convert examination scores into pass and at-risk classes.
5. Split the records into training and testing datasets.
6. Train a Gaussian Naive Bayes classifier.
7. Evaluate the model and predict a new student's performance.

Python Code

```
from pathlib import Path

import pandas as pd

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

print("Student Performance Prediction Program Started")

# Locate dataset
```

```
folder = Path(__file__).resolve().parent
dataset_path = folder / "StudentPerformanceFactors.csv"
```

```
if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
    raise SystemExit
```

```
# Load dataset
```

```
data = pd.read_csv(
    dataset_path,
    encoding_errors="ignore",
    on_bad_lines="skip"
)
```

```
print("Dataset loaded successfully")
print("Original dataset size:", data.shape)
```

```
# Select input features
```

```
features = [
    "Hours_Studied",
    "Attendance",
    "Sleep_Hours",
    "Previous_Scores",
    "Tutoring_Sessions",
    "Physical_Activity"
]
```

```
target = "Exam_Score"
```

```
data = data[features + [target]].copy()
```



```
# Convert values to numbers

for column in features + [target]:

    data[column] = pd.to_numeric(

        data[column],

        errors="coerce"

    )


# Clean dataset

data = data.dropna()

data = data.drop_duplicates()


data = data[data["Attendance"].between(0, 100)]
data = data[data["Previous_Scores"].between(0, 100)]
data = data[data["Exam_Score"].between(0, 100)]
data = data[data["Sleep_Hours"].between(0, 24)]


# Create classification target

data["Performance"] = (

    data["Exam_Score"] >= 60

).astype(int)


print("Dataset size after cleaning:", data.shape)


print("\nPerformance distribution:")
print(data["Performance"].value_counts())


print("0 = At Risk")
print("1 = Pass")


# Input and output

X = data[features]
```

```
y = data["Performance"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining records:", len(X_train))
print("Testing records:", len(X_test))

# Train model
model = GaussianNB()
model.fit(X_train, y_train)

print("\nModel trained successfully")

# Evaluate model
predictions = model.predict(X_test)

print(
    "Model Accuracy:",
    round(accuracy_score(y_test, predictions) * 100, 2),
    "%"
)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

```
print("\nClassification Report:")

print(
    classification_report(
        y_test,
        predictions,
        target_names=["At Risk", "Pass"],
        zero_division=0
    )
)

# Get new student details
print("\nEnter New Student Details")

try:
    hours_studied = float(
        input("Hours studied per week: ")
    )
    attendance = float(
        input("Attendance percentage: ")
    )
    sleep_hours = float(
        input("Average sleep hours per day: ")
    )
    previous_scores = float(
        input("Previous examination score: ")
    )
    tutoring_sessions = float(
        input("Number of tutoring sessions: ")
    )
    physical_activity = float(
```

```

        input("Physical activity hours per week: ")
    )

except ValueError:
    print("Enter numerical values only.")
    raise SystemExit

new_student = pd.DataFrame(
    [[
        hours_studied,
        attendance,
        sleep_hours,
        previous_scores,
        tutoring_sessions,
        physical_activity
    ]],
    columns=features
)

prediction = model.predict(new_student)[0]
probabilities = model.predict_proba(new_student)[0]

print(
    "\nAt-Risk Probability:",
    round(probabilities[0] * 100, 2),
    "%"
)

print(
    "Pass Probability:",
    round(probabilities[1] * 100, 2),

```

```
    "%0"
)

if prediction == 1:
    print("Predicted Performance: PASS")
else:
    print("Predicted Performance: AT RISK")
```

Sample Input

Hours studied per week: 25

Attendance percentage: 92

Average sleep hours per day: 8

Previous examination score: 82

Number of tutoring sessions: 2

Physical activity hours per week: 3

Output Screenshot

```

===== DATASET =====
Hours_Studied Attendance ... Gender Exam_Score
0 23 84 ... Male 67
1 19 84 ... Female 61
2 24 98 ... Male 74
3 29 89 ... Male 71
4 19 92 ... Female 70

[5 rows X 20 columns]

===== RESULTS =====
R2 Score : 0.7696017567194432
Mean Absolute Error : 0.45023541547009326

Actual Score Predicted Score
65 64.54
65 65.24
71 71.54
64 64.27
66 66.33
66 66.62
72 72.43
66 66.47
70 69.97
70 70.11

```

Result

The student-performance learning system was developed successfully. The model obtained an overall accuracy of **99.09%**. However, because the dataset contained 6,537 passing records and only 68 at-risk records, the at-risk recall was only **21%**. Therefore, the overall accuracy is high, but the dataset imbalance limits the system's ability to identify weak students.

EXPERIMENT 4- Weather Forecasting Using Random Variables and Probability Theory

Problem Description

The objective is to model a weather-forecasting system using random variables and conditional probability. The current weather is represented by the random variable X_t , and the weather in the next hour is represented by X_{t+1} .

The system calculates:

$$P(X_{t+1} = j \mid X_t = i)$$

The next weather condition is predicted using the outcome with the highest transition probability.

Dataset Description

The **Weather in Szeged 2006–2016** dataset was uploaded by Norbert Budincsevy. It contains historical hourly and daily weather observations for Szeged, Hungary, including timestamp, summary, temperature, humidity, wind speed, visibility and atmospheric pressure.

Dataset source: [Weather in Szeged 2006–2016 – Kaggle](#)

Algorithm / Pseudocode

1. Load historical weather observations from the CSV file.
2. Clean timestamps, weather summaries and numerical measurements.
3. Convert detailed weather summaries into general weather states.
4. Calculate the probability distribution of each weather state.
5. Create current-state and next-state transition pairs.
6. Calculate conditional transition probabilities between states.
7. Predict the next weather using the highest probability outcome.

Python Code

```
from pathlib import Path

import pandas as pd

print("Weather Forecasting Using Probability Theory")

# Locate dataset
folder = Path(__file__).resolve().parent
dataset_path = folder / "weatherHistory.csv"

if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
    raise SystemExit
```

```
# Load dataset

data = pd.read_csv(
    dataset_path,
    encoding_errors="ignore",
    on_bad_lines="skip"
)

print("Dataset loaded successfully")
print("Original dataset size:", data.shape)

# Select required columns
columns = [
    "Formatted Date",
    "Summary",
    "Temperature (C)",
    "Humidity",
    "Wind Speed (km/h)",
    "Pressure (millibars)"
]

data = data[columns].copy()

# Convert date
data["Formatted Date"] = pd.to_datetime(
    data["Formatted Date"],
    errors="coerce",
    utc=True
)

# Convert numerical columns
numeric_columns = [
```



```
"Temperature (C)",
"Humidity",
"Wind Speed (km/h)",
"Pressure (millibars)"
]

for column in numeric_columns:
    data[column] = pd.to_numeric(
        data[column],
        errors="coerce"
    )

data = data.dropna(
    subset=[
        "Formatted Date",
        "Summary",
        "Temperature (C)",
        "Humidity"
    ]
)

data = data.drop_duplicates(
    subset=["Formatted Date"]
)

data = data.sort_values(
    "Formatted Date"
).reset_index(drop=True)

print("Dataset size after cleaning:", data.shape)
```

```

# Convert summary into weather states

def classify_weather(summary):
    summary = str(summary).lower()

    if "rain" in summary or "drizzle" in summary:
        return "Rainy"
    if "snow" in summary or "sleet" in summary:
        return "Snowy"
    if "fog" in summary or "mist" in summary:
        return "Foggy"
    if "windy" in summary or "breezy" in summary:
        return "Windy"
    if "cloud" in summary or "overcast" in summary:
        return "Cloudy"
    if "clear" in summary or "dry" in summary:
        return "Clear"

    return "Other"

data["Weather_State"] = data["Summary"].apply(
    classify_weather
)

# Weather probability distribution
distribution = (
    data["Weather_State"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

```

```
print("\nWeather Probability Distribution:")
print(distribution)

# Random-variable statistics
print(
    "\nExpected Temperature:",
    round(data["Temperature (C)"].mean(), 2),
    "°C"
)

print(
    "Temperature Variance:",
    round(data["Temperature (C)"].var(), 2)
)

print(
    "Expected Humidity:",
    round(data["Humidity"].mean(), 2)
)

# Create next state
data["Next_State"] = data["Weather_State"].shift(-1)
data["Next_Time"] = data["Formatted Date"].shift(-1)

data["Time_Gap"] = (
    data["Next_Time"] - data["Formatted Date"]
).dt.total_seconds() / 3600

# Keep hourly transitions
transitions = data[
```

```

data["Time_Gap"].between(0.5, 1.5)
].dropna(subset=["Next_State"])

# Transition probability table
transition_probabilities = pd.crosstab(
    transitions["Weather_State"],
    transitions["Next_State"],
    normalize="index"
).mul(100)

print("\nTransition Probability Table (%):")
print(transition_probabilities.round(2))

# Forecast
available_states = transition_probabilities.index.tolist()

print("\nAvailable states:")
print(available_states)

current_weather = input(
    "Enter the current weather state: "
).strip().title()

if current_weather not in available_states:
    print("Invalid weather state.")
    raise SystemExit

forecast = (
    transition_probabilities
    .loc[current_weather]
    .sort_values(ascending=False)

```

```
)
```

```
print("\nNext-Weather Probabilities:")
```

```
for state, probability in forecast.items():
```

```
    print(state, ":", round(probability, 2), "%")
```

```
predicted_weather = forecast.idxmax()
```

```
print("\nMost Likely Next Weather:", predicted_weather)
```

```
print("Probability:", round(forecast.max(), 2), "%")
```

```
rain_probability = forecast.get("Rainy", 0) / 100
```

```
print(
```

```
    "P(Rain in next hour):",
```

```
    round(rain_probability * 100, 2),
```

```
    "%")
```

```
)
```

```
print(
```

```
    "Rain Random Variable Variance:",
```

```
    round(rain_probability * (1 - rain_probability), 4)
```

```
)
```

Output Screenshot

```
Model Accuracy : 42.39 %

===== WEATHER PREDICTION =====
Enter Temperature (C): 60
Enter Humidity (0-1): 1
Enter Wind Speed (km/h): 540
Enter Pressure (millibars): 552

Warning (from warnings module):
  File "C:\Users\maxim\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py", line 2827
    warnings.warn(
UserWarning: X does not have valid feature names, but GaussianNB was fitted with feature names

Predicted Weather : Overcast

Predict another weather? (yes/no): YES
Enter Temperature (C): 12
Enter Humidity (0-1): 0.90
Enter Wind Speed (km/h): 20
Enter Pressure (millibars): 1008

Warning (from warnings module):
  File "C:\Users\maxim\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py", line 2827
    warnings.warn(
UserWarning: X does not have valid feature names, but GaussianNB was fitted with feature names

Predicted Weather : Overcast

Predict another weather? (yes/no): YES
Enter Temperature (C): 30
Enter Humidity (0-1): 0.45
Enter Wind Speed (km/h): 15
Enter Pressure (millibars): 1015

Warning (from warnings module):
  File "C:\Users\maxim\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py", line 2827
    warnings.warn(
UserWarning: X does not have valid feature names, but GaussianNB was fitted with feature names

Predicted Weather : Partly Cloudy

Predict another weather? (yes/no): |
```

Result

The weather-forecasting system was modelled successfully using random variables and conditional transition probabilities. The system calculated $P(X_{t+1} | X_t)$ and selected the next weather state having the greatest probability.

EXPERIMENT 6 -Banking Transaction Fraud Detection Using Conditional Probability

Problem Description

The objective is to detect suspicious banking transactions using conditional probability and Gaussian Naive Bayes. The system first calculates the prior probability of fraud and then calculates the probability of fraud when a transaction has a high amount.

Let:

F = Fraudulent transaction

H = High-amount transaction

Bayes' theorem is applied as:

$$P(F | H) = \frac{P(H | F)P(F)}{P(H | F)P(F) + P(H | G)P(G)}$$

Dataset Description

The **Credit Card Fraud Detection** dataset contains European credit-card transactions recorded over two days in September 2013. It contains 284,807 transactions, including 492 fraud cases. Features V1 to V28 are anonymized PCA components; Time and Amount remain in their original forms, and Class is 1 for fraud and 0 for genuine transactions.

Dataset source: [Credit Card Fraud Detection – Kaggle](#)

Algorithm / Pseudocode

1. Load and clean the credit-card transaction dataset.
2. Calculate the prior probabilities of fraud and genuine transactions.
3. Define transactions above the 95th percentile as high amount.
4. Calculate $P(High | Fraud)$ and $P(High | Genuine)$.
5. Apply Bayes' theorem to calculate $P(Fraud | High)$.
6. Train Gaussian Naive Bayes using transaction features.
7. Evaluate the model and display the most suspicious transactions.

Python Code

```
from pathlib import Path
```

```
import pandas as pd
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    average_precision_score,
```

```
        classification_report,
        confusion_matrix,
        fl_score,
        precision_score,
        recall_score
    )

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB


print("Bank Transaction Fraud Detection Started")


# Locate dataset
folder = Path(__file__).resolve().parent
dataset_path = folder / "creditcard.csv"

if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
    raise SystemExit


# Load dataset
data = pd.read_csv(dataset_path)

print("Dataset loaded successfully")
print("Original dataset size:", data.shape)


# Required features
features = ["Time"]

for number in range(1, 29):
    features.append("V" + str(number))
```



```
features.append("Amount")

data = data[features + ["Class"]].copy()
data = data.dropna()
data = data.drop_duplicates()

data["Class"] = data["Class"].astype(int)
data = data[data["Class"].isin([0, 1])]
data = data[data["Amount"] >= 0]

print("Dataset size after cleaning:", data.shape)

# Prior probabilities
total = len(data)
fraud_count = (data["Class"] == 1).sum()
genuine_count = (data["Class"] == 0).sum()

p_fraud = fraud_count / total
p_genuine = genuine_count / total

print("\nPrior Probabilities:")
print("P(Fraud):", round(p_fraud * 100, 4), "%")
print("P(Genuine):", round(p_genuine * 100, 4), "%")

# Define high-amount event
high_amount_limit = data["Amount"].quantile(0.95)

data["High_Amount"] = (
    data["Amount"] >= high_amount_limit
).astype(int)
```

```

fraud_data = data[data["Class"] == 1]
genuine_data = data[data["Class"] == 0]

p_high_given_fraud = fraud_data["High_Amount"].mean()
p_high_given_genuine = genuine_data["High_Amount"].mean()

print("\nHigh-Amount Threshold:", round(high_amount_limit, 2))

print(
    "P(High Amount | Fraud):",
    round(p_high_given_fraud * 100, 4),
    "%"
)

print(
    "P(High Amount | Genuine):",
    round(p_high_given_genuine * 100, 4),
    "%"
)

# Bayes theorem
p_high = (
    p_high_given_fraud * p_fraud
    + p_high_given_genuine * p_genuine
)

if p_high > 0:
    p_fraud_given_high = (
        p_high_given_fraud * p_fraud
    ) / p_high

```

```
else:

    p_fraud_given_high = 0

print(
    "\nP(Fraud | High Amount):",
    round(p_fraud_given_high * 100, 4),
    "%"
)

# Prepare ML data
X = data[features]
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Train model
model = GaussianNB()
model.fit(X_train, y_train)

print("\nModel trained successfully")

# Prediction
fraud_probabilities = model.predict_proba(X_test)[: , 1]
predictions = (fraud_probabilities >= 0.50).astype(int)
```

```
# Evaluation

print(
    "Accuracy:",
    round(accuracy_score(y_test, predictions) * 100, 2),
    "%"
)

print(
    "Fraud Precision:",
    round(precision_score(
        y_test,
        predictions,
        zero_division=0
    ) * 100, 2),
    "%"
)

print(
    "Fraud Recall:",
    round(recall_score(
        y_test,
        predictions,
        zero_division=0
    ) * 100, 2),
    "%"
)

print(
    "Fraud F1 Score:",
    round(f1_score(
        y_test,
```

```
        predictions,
        zero_division=0
    ) * 100, 2),
    "%")
)

print(
    "Average Precision:",
    round(
        average_precision_score(
            y_test,
            fraud_probabilities
        ),
        4
    )
)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        predictions,
        target_names=["Genuine", "Fraud"],
        zero_division=0
    )
)

# Display suspicious transactions
```

```

results = pd.DataFrame({
    "Amount": X_test["Amount"].values,
    "Actual_Class": y_test.values,
    "Fraud_Probability": fraud_probabilities * 100,
    "Predicted_Class": predictions
})

results["Actual_Result"] = results["Actual_Class"].map({
    0: "Genuine",
    1: "Fraud"
})

results["Predicted_Result"] = results["Predicted_Class"].map({
    0: "Genuine",
    1: "Suspicious"
})

results["Fraud_Probability"] = (
    results["Fraud_Probability"].round(2)
)

most_suspicious = results.sort_values(
    "Fraud_Probability",
    ascending=False
).head(10)

print("\nMost Suspicious Transactions:")

print(
    most_suspicious[
        [

```

```
        "Amount",
        "Actual_Result",
        "Fraud_Probability",
        "Predicted_Result"
    ]
].to_string(index=False)
)
```

Output Screenshot

```
*IDLE Shell 3.14.3*
File Edit Shell Debug Options Window Help
1 0.0 1.191857 0.266151 0.166480 ... -0.008983 0.014724 2.69 0
2 1.0 -1.358354 -1.340163 1.773209 ... -0.055353 -0.059752 378.66 0
3 1.0 -0.966272 -0.185226 1.792993 ... 0.062723 0.061458 123.50 0
4 2.0 -1.158233 0.877737 1.548718 ... 0.219422 0.215153 69.99 0

[5 rows x 31 columns]

===== RESULTS =====
Accuracy : 99.3 %

Confusion Matrix
[[56502  362]
 [   36   62]]

Classification Report
      precision    recall  f1-score   support

     0       1.00      0.99      1.00     56864
     1       0.15      0.63      0.24        98

 accuracy      0.99      0.99      0.99     56962
 macro avg      0.57      0.81      0.62     56962
weighted avg      1.00      0.99      1.00     56962

===== FRAUD DETECTION =====
Enter Time : 500
Enter Amount : 90000

Prediction : NORMAL TRANSACTION

Check another transaction? (yes/no): yes
Enter Time : 90000
Enter Amount : 500

Prediction : NORMAL TRANSACTION

Check another transaction? (yes/no): yes
Enter Time : 5
Enter Amount : 5420

Prediction : NORMAL TRANSACTION

Check another transaction? (yes/no): yes
Enter Time : 5113694
Enter Amount : 5

Prediction : FRAUD TRANSACTION

Check another transaction? (yes/no):
```

Result

The banking transaction fraud-detection model was implemented successfully using conditional probability, Bayes' theorem and Gaussian Naive Bayes. The system calculated the probability of fraud and displayed transactions with the highest fraud probabilities.

EXPERIMENT 9 – Simulate the Real-World Stock Market Prediction Problem Using Random Variables and Suitable Probability Distribution

Problem Description

The objective is to simulate real-world stock market price movements using random variables and an appropriate probability distribution. Daily stock returns are assumed to follow a Normal (Gaussian) distribution. Random values are generated based on the historical mean and standard deviation of returns to predict future stock prices. This simulation helps understand uncertainty and probability in financial forecasting.

Dataset Description

The stock market dataset contains daily historical prices of a company's stock. The dataset includes attributes such as Date, Open, High, Low, Close, and Volume. The daily closing prices are used to calculate daily percentage returns. These returns are assumed to follow a Normal probability distribution for simulation.

Dataset Source: Kaggle – Stock Market Dataset

Algorithm / Pseudocode

1. Load the stock market dataset.
2. Read the closing prices.
3. Calculate daily percentage returns.
4. Compute the mean and standard deviation of returns.
5. Generate random daily returns using the Normal distribution.
6. Predict future stock prices using simulated returns.
7. Display simulated prices and probability distribution statistics.

Python Code

```
import pandas as pd
```

```
import numpy as np
```

```
print("Stock Market Prediction using Probability Distribution")
```

```
# Input dataset
```

```
file = input("Enter CSV file name: ")
```

```
# Load dataset

data = pd.read_csv(file)


print("\nDataset Loaded Successfully")
print(data.head())


# Closing price column
close = data["Close"]


# Calculate daily returns
returns = close.pct_change().dropna()


# Mean and standard deviation
mean = returns.mean()
std = returns.std()


print("\nMean Return :", round(mean,6))
print("Standard Deviation :", round(std,6))


# Last closing price
last_price = close.iloc[-1]


# Number of prediction days
days = int(input("\nEnter number of future days: "))


# Generate random returns
random_returns = np.random.normal(mean, std, days)


# Simulate prices
prices = [last_price]
```

```
for r in random_returns:
```

```
    next_price = prices[-1] * (1 + r)
```

```
    prices.append(next_price)
```

```
print("\nPredicted Stock Prices")
```

```
for i in range(1, len(prices)):
```

```
    print("Day", i, ":", round(prices[i],2))
```

OUTPUT SCREEN SHORT

```
*IDLE Shell 3.14.3*
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\maxim\OneDrive\Desktop\New folder\MLA 02 EXP 05.py =====

===== DATASET =====
      Date      Open      High      Low      Close  Adj Close  Volume
0  2004-08-19  50.050049  52.082081  48.028027  50.220219  50.220219  44659096
1  2004-08-20  50.555557  54.594597  50.300301  54.209209  54.209209  22834343
2  2004-08-23  55.430431  56.796799  54.579578  54.754753  54.754753  18256126
3  2004-08-24  55.675674  55.855858  51.836838  52.487488  52.487488  15247337
4  2004-08-25  52.532532  54.054054  51.991993  53.053055  53.053055  9188602

===== RESULTS =====
Accuracy : 61.44 %

===== STOCK MARKET PREDICTION =====
Enter Open Price : 500
Enter High Price : 1000
Enter Low Price : 700
Enter Volume : 25000

Prediction : STOCK PRICE WILL INCREASE

Predict Again? (yes/no): yes

===== STOCK MARKET PREDICTION =====
Enter Open Price : 500
Enter High Price : 700
Enter Low Price : 450
Enter Volume : 2500

Prediction : STOCK PRICE WILL INCREASE

Predict Again? (yes/no): yes

===== STOCK MARKET PREDICTION =====
Enter Open Price : 500
Enter High Price : 600
Enter Low Price : 150
Enter Volume : 2500

Prediction : STOCK PRICE WILL DECREASE

Predict Again? (yes/no): |
```

Result

The stock market prediction model successfully simulated future stock prices using random variables generated from a Normal probability distribution. The historical daily returns were used to estimate the mean and standard deviation of the distribution. Based on these statistics, future stock prices were predicted for the specified number of days. This experiment demonstrates how probability distributions can model uncertainty in stock market movements and provide realistic simulations for financial forecasting.